HDF5 Audit Report

[NN]

Very abstract...

hello

Acknowledgments: This material is based upon work supported by the U.S. National Science Foundation under Federal Award No. 2534078. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Revision History

Version Number	Date	Comments
v1	July 4, 2026	Initial draft released for internal review.
v2	August 1, 2026	Public release.

Contents

1. Introduction **5**
1.1. Purpose . . . 5
1.2. Scope . . . 5
1.2.1. Safety . . . 5
1.2.2. Security . . . 5
1.2.3. Privacy . . . 5

2. Motivation **5**
2.1. Problem Statement . . . 5
2.2. Goals . . . 5

3. HDF5 Core Library and File Format **7**
3.1. Accidents Waiting to Happen – HDF5 Safety . . . 7
3.2. Verify and Verify – HDF5 Security . . . 7
3.3. Footprints, Fingerprints, Controls – HDF5 Privacy . . . 8
3.4. HDF5 Tools . . . 8
3.5. Audit Findings and Mitigation Priorities . . . 8

4. HDF5 Extensions **23**
4.1. Overview . . . 23
4.2. Virtual Object Layer (VOL) Connectors . . . 23
4.3. Filters Plugins . . . 23
4.4. Virtual File Drivers (VFD) . . . 23
4.5. Audit Findings and Mitigation Priorities . . . 23

5. HDF5 Wrappers and Language Bindings **24**
5.1. Overview . . . 24
5.2. Python . . . 24
5.3. Java . . . 24
5.4. Audit Findings and Mitigation Priorities . . . 24

6. HDF5 in the Software Supply Chain **25**
6.1. Upstream Dependencies 25
6.2. Downstream Dependencies 26
6.3. Audit Findings and Mitigation Priorities 29

7. Independent HDF5 Implementations **31**
7.1. Overview 31
7.2. Audit Findings and Mitigation Priorities 31

8. Long-term Data Accessibility and Interpretability **32**
8.1. Audit Findings and Mitigation Priorities 32

9. Resources **33**

References **34**

A. CVE History Summary **37**

B. Software Supply Chain Details **42**

1. Introduction

1.1. Purpose

1.2. Scope

Out of scope:

- PHDF5
- HDFView
- HSDS
- AI

1.2.1. Safety

1.2.2. Security

1.2.3. Privacy

2. Motivation

2.1. Problem Statement

2.2. Goals

- Emphasize broader applicability for comparable OSEs

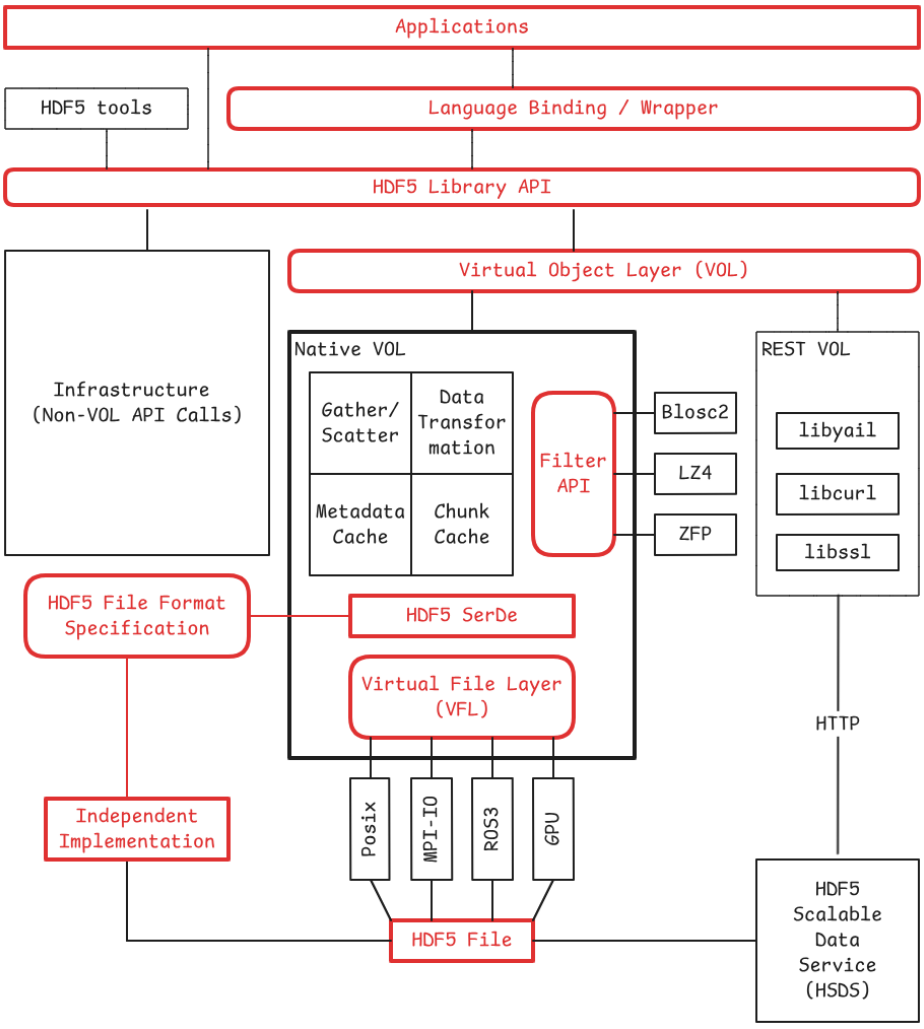

Figure 1 – HDF5 vulnerability sites (interfaces: rounded rectangles)

3. HDF5 Core Library and File Format

3.1. Accidents Waiting to Happen – HDF5 Safety

- Disk full [5]
- File corruption [3]
- Long-term data accessibility and interpretability, see section 8

3.2. Verify and Verify – HDF5 Security

- Integer arithmetic
- File format parsing issues
- Malicious plugins, see section 4

A CVE history summary is included in Appendix A. The list mixes true HDF5-library issues, removed-tool issues, duplicate/rejected/not-HDF5 entries, and entries with no commit URL. The main pattern is not "many unrelated bugs"; it is repeated failure to validate file-derived structure sizes, offsets, counts, message lengths, continuation chunks, datatype metadata, or filter parameters before using them in low-level C memory operations. The 2025 entries alone show repeated heap overflows, use-after-free, null dereference, memory leak, and resource-consumption defects across object headers, file-space metadata, type conversion, filters, and metadata cache code.

The fix commentary confirms that several CVEs were symptoms of corrupted metadata not being rejected early. (Examples: the file-space page-size fix added a sanity check immediately after reading `page_size` from the file; the object-header-continuation fix rejects zero continuation chunk sizes; the self-referential continuation-message fix checks expected versus actual deserialized object-header chunks; and the cache-image fix added buffer-size arguments, overflow checks, input validation, and cleanup.)

The strategic root-cause categories are tighter than the CVE count suggests, and are summarized in Table 1.

Table 1 – Strategic root-cause categories in the HDF5 CVE history

Root-cause category	What it means for HDF5
unchecked file-derived sizes/counts/offsets	The dominant class. HDF5 metadata controls object layout, heap blocks, chunks, dataspaces, filters, links, and type descriptors. When those fields are malformed, C code later trusts them.
inconsistent internal bookkeeping	Attribute tables, cache images, continuation chunks, and free lists often needed separate "allocated capacity" versus "actual count" tracking.
decode/encode asymmetry	Several issues happen because decode did not reject bad messages, then encode/flush later assumed valid native structures.
arithmetic precondition failures	Divide-by-zero and multiplication-overflow checks were missing in chunking, layout, b-tree, filter, and dataspace paths.
unsafe cleanup/lifetime paths	Use-after-free, double-free, leaks, and null dereferences show that malformed input often breaks unwinding logic as much as the primary decode logic.
legacy tool attack surface	`gif2h5`, HDF to GIF conversion, `h5dump`, and `h5repack` appear repeatedly. Removal of `gif2h5` was a structural mitigation, not just a patch.
incomplete triage metadata	Several entries have `commit_url:null`, "not filed against HDF5," "cannot reproduce," "not applicable," or tentative confidence. Those should not be counted as reviewed HDF5 fixes without qualification.

3.3. Footprints, Fingerprints, Controls – HDF5 Privacy

3.4. HDF5 Tools

Not strictly part of the core library, but the tools are an important part of the ecosystem and have their own set of issues.

3.5. Audit Findings and Mitigation Priorities

The blunt conclusion: HDF5's CVE history is mostly malformed-file parsing pressure against a complex binary metadata machine. The fix direction should

not be "patch each crash." It should be systematic metadata validation at decode boundaries, explicit invariants for each on-disk structure, fuzz corpora tied to invariants, and hard separation between parser acceptance and internal object construction.

Goal: HDF5 should be able to parse any byte stream as hostile input and either reject it cleanly or produce a validated internal object graph. Anything else is security debt.

Implementation program: define parser invariants first, harden decode boundaries, then make fuzzing, CI gates, and release governance enforce them.

The next priority is parser normalization: decode file bytes into a validated intermediate form before constructing internal objects, so malformed metadata cannot reach trusted C paths.

Our goal should be: *make malformed HDF5 files boring*. Every corrupted file should terminate in a controlled `H5E_BADVALUE`, `H5E_OVERFLOW`, `H5E_CORRUPT`, or `H5E_RESOURCE` path. No segfaults. No assertions. No leaks. No undefined behavior. No late-stage encode/flush crash from a bad decode.

The current CVE list includes many issues in `H5FS`, `H5C`, `H5O`, `H5T`, `H5Z`, and vector-copy paths, with repeated heap overflows, use-after-free, null dereference, and resource-consumption outcomes. HDF5's own format documentation explains why this happens: a file is a graph of groups, datasets, and named datatypes; on disk it is represented through lower-level structures such as the superblock, B-tree nodes, object headers, global heaps, local heaps, and free-space metadata. That means the library is parsing a dense graph of mutually referential, *attacker-controlled* metadata.

Priority 0: Stop closing CVEs as "fixed" unless the class is killed

Make this rule immediate:

If the fix is not a structural change that eliminates the root cause, then it is not a fix.

For every new CVE or crash:

Table 2 – Required artifacts for closing HDF5 CVEs or crash reports

Required artifact	Purpose
crashing input	becomes a permanent corpus seed
minimized input	used in quick PR regression
root-cause category	maps to a known invariant class
invariant update	prevents sibling bugs
sanitizer regression	proves no crash, no leak, no UAF
negative test	checks controlled failure code
reviewer checklist update	prevents reintroduction

This is the first cultural shift. A patch that only adds a local `if` is tolerated for embargoed fixes, but the issue remains open as a **class bug** until the invariant framework catches sibling cases.

Recent commits show the right direction, but too locally. One fix added a sanity check immediately after reading file-space page size from the file and used the same fuzzer to verify that the previous assertion became a corrupted-file error. [10] Another hardened `H5C__reconstruct_cache_entry()` by adding buffer-size tracking, overflow checks, input validation, and safe cleanup. [9] Those are exactly the moves to systematize.

Priority 1: Build an invariant registry for the file format

Create a versioned document and machine-readable metadata file, for example:

```
security/invariants/
  superblock.yml
  object_header.yml
```

```
  dataspace.yml
  datatype.yml
  layout.yml
  filters.yml
  heaps.yml
  free_space.yml
  links.yml
  cache_image.yml
```

Each entry should define:

```
id: H5O-CONT-001
component: object_header
structure: continuation_message
condition: continuation_chunk_size > 0
failure: H5E_CORRUPT
checked_at:
  - H5O__cont_decode
  - H5O__chunk_deserialize
fuzz_targets:
  - fuzz_object_header
cves:
  - CVE-2025-6856
  - CVE-2025-6816
```

Start with these invariant families.

Table 3 – Invariant families to enforce first

Area	Invariants to enforce first
superblock	offset size and length size are valid; base address, EOF address, driver info address, and root object address are inside the file or explicitly undefined; EOF does not wrap
object headers	message length fits inside chunk; continuation chunks are nonzero; continuations do not self-reference; continuations are acyclic; message flags match message type; unknown messages cannot be cast into known message structures

Table 3 – Invariant families to enforce first, continued

Area	Invariants to enforce first
free-space meta-data	page size is nonzero and below max; section size fits page/file bounds; serialized section count matches decoded section count
heaps	heap object offset and size are inside heap; free-list blocks do not overlap; free-block list is acyclic; no allocation uses unchecked `count*size`
dataspaces	rank is bounded; dimension product is checked; selected point count is checked; hyperslab stride, count, block, and start do not overflow
datatypes	storage size is nonzero; precision fits storage size; member offsets fit compound size; string length is bounded; vlen/ref types cannot escape validation
layouts	compact data size equals `dataspaceelements*datatypesize`; chunk dimensions are nonzero; chunk product is checked; chunk index references are inside file bounds
filters	parameter count is bounded; decompressed size is bounded; scale-offset precision is validated; filter output ratio cannot exceed configured resource limits
cache image	cache-entry count, address, size, and alignment are bounded; decoding consumes exactly the expected number of bytes; cleanup is safe after partial decode
tools	string rendering is length-bounded; conversion tools never trust input dimensions; tool-only parsers use the same checked arithmetic wrappers as the library

This registry becomes the source of truth for code review, tests, fuzzing, and documentation.

Priority 2: Introduce a two-phase parser boundary

Right now, too many paths appear to do this:

```
file bytes -> native internal object
           -> later code assumes validity
```

Change the architecture to:

```
file bytes -> bounded raw decode
           -> semantic validation
           -> native object construction
```

Concrete rule: *No public or internal runtime object is constructed from file metadata until the corresponding validator has passed.*

Implement the pattern by component:

```
herr_t H5O_msg_decode_raw(const uint8_t *buf, size_t buf_size,
↪  H5O_msg_raw_t *raw);
herr_t H5O_msg_validate(const H5F_t *f, const H5O_msg_raw_t *raw,
↪  H5O_validation_ctx_t *ctx);
herr_t H5O_msg_construct(const H5O_msg_raw_t *raw, H5O_msg_t *out);
```

Do not let decode functions allocate complex structures before basic bounds are known. A raw decoded object should hold values, offsets, and spans, not trusted pointers into long-lived internal state.

All decode functions that consume file bytes should take one of these forms:

```
herr_t decode(const H5F_t *f, const uint8_t *buf, size_t buf_size,
↪  out_t *out);

herr_t decode_cursor(H5_decode_cursor_t *cur, out_t *out);
```

Ban this pattern in security-sensitive decode code:

```
const uint8_t **pp
```

unless there is also an explicit `p_end` or remaining-size counter. The cache-image fix already moved in this direction by adding a buffer-size argument and bounds checks. [9]

Priority 3: Centralize checked arithmetic and allocation

Most parser bugs become exploitable when metadata reaches allocation, copy, or pointer arithmetic. Add a small, mandatory checked-arithmetic layer.

Example API:

```
herr_t H5_checked_add_hsize(hsize_t a, hsize_t b, hsize_t *out);
herr_t H5_checked_mul_hsize(hsize_t a, hsize_t b, hsize_t *out);
herr_t H5_checked_addr_add(haddr_t base, hsize_t len, haddr_t eoa,
↪  haddr_t *end);
herr_t H5_checked_alloc_size(size_t count, size_t elem_size, size_t
↪  *bytes);
```

Then make these rules review-blocking:

Table 4 – Review-blocking parser arithmetic and allocation patterns

Ban	Replacement
`malloc(n*size)`	`H5MM_calloc_checked(n,size)`
`addr+size`	`H5_checked_addr_add(addr, size,eoa,&end)`
`memcpy(dst,src,n)` from file-derived size	`H5_checked_memcpy(dst,dst_size,src,src_size,n)`
pointer increment without `p_end`	cursor-based decode
raw recursion through object graph	bounded traversal context

The 2024 security commit added multiplication-overflow checks for dataset storage size and checks against file bounds. [11] That pattern should become a library-wide rule, not a local repair.

Priority 4: Create security profiles for file opening

Add explicit modes. Do not rely on users to infer safe behavior.

```
typedef enum {
    H5_SECURITY_PROFILE_LEGACY,
    H5_SECURITY_PROFILE_TRUSTED_FAST,
    H5_SECURITY_PROFILE_UNTRUSTED_STRICT,
    H5_SECURITY_PROFILE_FORENSIC
} H5_security_profile_t;
```

Suggested semantics:

Table 5 – Security profile semantics

Profile	Behavior
`legacy`	current compatibility bias
`trusted_fast`	normal validation, generous resource limits
`untrusted_strict`	strict metadata validation, plugin loading disabled unless allowlisted, resource limits enforced
`forensic`	never repairs silently, reports all structural anomalies, best-effort inspection without unsafe traversal

Expose it through file-access property lists:

```
H5Pset_security_profile(fapl, H5_SECURITY_PROFILE_UNTRUSTED_STRICT);
H5Pset_max_metadata_bytes(fapl, 256 * 1024 * 1024);
H5Pset_max_object_header_chunks(fapl, 1024);
H5Pset_max_btree_depth(fapl, 64);
H5Pset_max_filter_expansion_ratio(fapl, 100);
H5Pset_plugin_policy(fapl, H5_PLUGIN_POLICY_DISABLED);
```

This matters because HDF5 supports dynamically loaded plugins, and the runtime API can enable or disable plugin classes; the docs also state that `HDF5_PLUGIN_PRELOAD=::` disables all dynamic plugins. Turn that capability into a first-class security policy, not an environment-variable trick.

Priority 5: Fuzz by structure, not just by whole-file crashes

Whole-file fuzzing is necessary but insufficient. It finds symptoms. Structure-aware fuzzing finds families.

Build fuzz targets in this order:

Table 6 – Structure-aware fuzzing priorities

Order	Fuzz target	Why first
1	object-header message decoder	many paths flow through it
2	datatype decoder and converter	high exploitability and complex nested metadata
3	dataspace and selection decoder	rank, count, hyperslab, and point math
4	layout and chunk index decoder	storage-size, address, and chunk-product risks
5	free-space and local/global heap decoders	repeated historical overflow and list-corruption class
6	filter pipeline and scale-offset filter	compression/decompression boundary risk
7	metadata cache image reconstruction	high complexity and recent CVE pattern
8	`h5dump`/`h5repack` rendering paths	tool paths often bypass library assumptions

Use three fuzzing layers:

Table 7 – Fuzzing layers

Layer	Description
byte-level fuzzing	current randomized mutation against APIs
structure-aware mutation	valid superblock with corrupted object headers, valid object header with bad continuation, valid datatype with bad member offset
invariant-guided generation	generate files that violate exactly one invariant at a time

OSS-Fuzz already supports libFuzzer, AFL++, Honggfuzz, Centipede, sanitizers, and distributed execution for C/C++ projects. Use OSS-Fuzz for long-running public coverage and ClusterFuzzLite or local CI for PR-level checks.

Table 8 – Fuzzing acceptance criteria

Stage	Gate
PR	run minimized CVE corpus under `ASan`/`UBSan`/`LSan`
nightly	run structure fuzzers for core parsers
weekly	report coverage by component, not just total line coverage
release candidate	72-hour fuzz soak on changed parser areas
CVE closure	crashing input added to corpus and mapped to invariant

Priority 6: Make malformed-input regression cheap

Create this repository layout:

```
test/security/
  corpus/
    cve/
      CVE-2025-7069/
        input.h5
```

```
        expected.yml
      CVE-2025-6269/
        input.h5
        expected.yml
    invariants/
      H5O-CONT-001/
      H5T-SIZE-002/
  harnesses/
    fuzz_object_header.c
    fuzz_datatype.c
    fuzz_dataspace.c
    fuzz_layout.c
    fuzz_filters.c
  expected_errors/
    corrupt_file.yml
```

Each expected.yml should include:

```
expected:
  exit: controlled_error
  allowed_errors:
    - H5E_CORRUPT
    - H5E_BADVALUE
    - H5E_OVERFLOW
forbidden:
  - SIGSEGV
  - SIGABRT
  - leak
  - use_after_free
  - timeout
invariant:
  - H5FS-PAGE-001
```

This gives maintainers a fast way to prove that old crashes remain dead.

Priority 7: Add a security-review checklist that blocks risky parser changes

Every PR touching decode, encode, object lifecycle, filters, tools, or plugin loading should answer:

Table 9 – Security-review checklist for parser-adjacent changes

Question	Required answer
Does this read file-derived bytes?	list buffer and bounds source
Does this allocate using file-derived values?	show checked multiplication
Does this copy memory using file-derived sizes?	show source and destination bounds
Does this traverse links, chunks, heaps, B-trees, or continuations?	show cycle and depth bounds
Does this construct native objects before validation?	explain why or refactor
Does this add or change cleanup paths?	show partial-init safety
Does this invoke plugins or filters?	show policy interaction
Does this alter error handling?	show malformed-input test

Add automated enforcement where possible:

```
fail PR if new decode function lacks size parameter
fail PR if malloc(count * size) appears outside checked wrapper
fail PR if memcpy appears in parser code without checked wrapper annotatio
fail PR if new parser file lacks fuzz target registration
fail PR if new object traversal lacks max-depth or visited-set logic
```

This will annoy contributors at first. Good. That friction is cheaper than post-release CVEs.

Priority 8: Separate compatibility from safety

HDF5's backward compatibility is valuable, but it creates a security trap: old, permissive parsing keeps malformed legacy structures alive.

Implement this policy:

Table 10 – Compatibility and safety profile policy

Case	Behavior
valid legacy file	accepted
ambiguous legacy construct	accepted only in `legacy` or `trusted_fast`; warning in `forensic`; rejected in `untrusted_strict`
structurally invalid file that old versions tolerated	rejected in all non-legacy profiles
parser repair needed	repair only through explicit tool, never silently during normal open
unknown message with unsafe flags	rejected unless profile explicitly allows preservation

The point is not to break legitimate archives. The point is to stop treating corrupted metadata as compatibility.

Priority 9: Harden tools as separate attack surfaces

Do not assume `h5dump`, `h5repack`, converters, and test utilities are less important. They are often the first thing users run on untrusted files.

Actions:

Table 11 – Tool hardening actions

Tool class	Change
inspection tools	use `H5_SECURITY_PROFILE_FORENSIC` by default

Table 11 – Tool hardening actions, continued

Tool class	Change
conversion tools	use `H5_SECURITY_PROFILE_UNTRUSTED_STRICT` by default
rendering paths	bounded strings only
legacy converters	disable by default; build behind explicit `HDF5_ENABLE_LEGACY_UNSAFE_TOOLS=ON`
plugin-dependent tools	warn or require allowlist

A dangerous file should not become more dangerous because the user tried to inspect it.

Priority 10: Plugin policy and signed extensions

Plugins are a different risk class. They are executable code loaded at runtime. HDF5's plugin system supports dynamically loaded filter plugins and lets users implement custom filters. That is useful, but it must not be invisible in untrusted mode.

Implement:

```
default profile:
  trusted_fast: existing plugin behavior
  untrusted_strict: plugins disabled unless allowlisted
  forensic: plugins disabled; report missing filter
```

Add a plugin manifest:

```
plugin_id: 32008
name: blosc
version: 1.0.0
sha256: ...
signer: ...
allowed_profiles:
```

```
  - trusted_fast
  - untrusted_strict
capabilities:
  - filter_decode
limits:
  max_expansion_ratio: 100
  max_alloc_bytes: 1073741824
```

Do not start with a perfect PKI. Start with allowlisted hashes and paths. Then add signatures. The signed-plugin work matters, but without resource limits a signed buggy plugin can still crash the process.

Priority 11: Supply-chain controls after parser controls

Supply-chain work matters, but it is second-order relative to malformed-file parsing. Do it in parallel only after the parser program has owners.

Minimum controls:

Table 12 – Minimum supply-chain controls

Control	Target
OpenSSF Scorecard	weekly scheduled run, alerts in security tab
CodeQL	required for PRs touching C/C++ parser surfaces
pinned GitHub Actions	no floating third-party action refs
SBOM	release artifacts include source and binary SBOMs
signed release artifacts	signatures plus checksums
SLSA provenance	release builds produce verifiable provenance

OpenSSF Scorecard has an official GitHub Action, and public repositories can use it without cost. The SLSA GitHub generator provides tooling for SLSA provenance on GitHub Actions, including provenance that users can verify to understand how an artifact was built.

Priority 12: Metrics that force the right behavior

Use metrics that measure class elimination, not activity.

Table 13 – Security program metrics

Metric	Target
CVE corpus pass rate under ASan/UBSan/LSan	100%
parser decode functions with explicit buffer size or cursor	100%
parser allocations through checked wrappers	100%
parser memcpy/memmove through checked wrappers or reviewed exemption	100%
high-risk structures with invariant files	100%
high-risk structures with fuzz target	100%
new CVEs that add invariant updates	100%
time from new crash to minimized corpus seed	under 48 hours
time from minimized seed to invariant classification	under 5 business days
sanitizer-clean nightly fuzz runs	sustained
untrusted profile test coverage	every release

Track this in a public dashboard. Not because dashboards fix bugs. They stop the team from pretending that “more tests” is a plan.

4. HDF5 Extensions

4.1. Overview

Supply chain-related issues are discussed in more detail in Section 6.

4.2. Virtual Object Layer (VOL) Connectors

4.3. Filters Plugins

4.4. Virtual File Drivers (VFD)

4.5. Audit Findings and Mitigation Priorities

5. HDF5 Wrappers and Language Bindings

5.1. Overview

Complete:

- C#
- Java
- Python
- JavaScript
- Rust

Partials…

5.2. Python

- CVE-2025-995 [22]
- CVE-2026-0897 [23]
- CVE-2026-1669 [24]
- CVE-2026-2492 [18]

5.3. Java

5.4. Audit Findings and Mitigation Priorities

6. HDF5 in the Software Supply Chain

- HDF5 2.0.0 source-artifact integrity check [31]
- SBOM

6.1. Upstream Dependencies

This subsection summarizes external software that can be required by an HDF5 build. It starts with the default serial C library and is organized by build option, because the default serial C library has a much smaller dependency set than feature-complete, testing, packaging, or developer builds. More details are provided in Appendix B.

HDF5's CMake build can either locate dependencies that are already installed or, for selected filter-related projects, add external source trees to the build with `FetchContent`. See table 16 for details.

The baseline HDF5 build has no mandatory external dependencies. The core library and C API tests can be built and run without any additional software. See table 17 for details.

The HDF5 build system has options that trigger additional dependencies when enabled. For example, enabling parallel HDF5 requires an MPI implementation, while enabling Fortran support requires a Fortran compiler. See table 18 for details.

Enabling filter plugins or the plugin project can introduce additional filter-library dependencies. The HDF5 cache defaults include entries for bitgroom, bitround, bitshuffle, Blosc, Blosc2, bzip2, fpzip, JPEG, LZ4, LZF, SZ, ZFP, and Zstandard. Those libraries are controlled by the plugin project's own options when `hdf5_plugins` is built or found. See table 19 for details.

Enabling VFDs for ROS3, HDFS, or other storage systems can introduce dependencies on external libraries and credentials. The same applies to VOL connectors. These are not filter dependencies, but they can affect the security and robustness of HDF5 file access, especially for third-party data. See table 20 for details.

Additional dependencies are required for optional tools, tests, and security features. For example, the `hdf5_security` target depends on OpenSSL when built with `HDF5_ENABLE_SECURITY_FEATURES`. See table 21 for details.

Finally, developer builds and packaging scripts have their own dependencies. These include code linters, formatters, static analyzers, documentation tools, and packaging tools. These dependencies are not required for a default HDF5 build, but they are relevant for the supply-chain risk of development and release processes. For example, if a security vulnerability is introduced in a packager or source formatter, it could affect the integrity of HDF5 releases even if the vulnerability is not present in the HDF5 source code itself. See table 22 for details.

Several default upstream source locations are encoded in `config/CacheURLs.cmake` for dependencies that HDF5 can fetch or whose values are passed to external filter-plugin builds. See table 23 for details.

6.2. Downstream Dependencies

For this audit, downstream dependencies are projects that compile against, dynamically load, package, or define data conventions on top of HDF5. The HDF5 ecosystem is much larger than any static list, so the most important downstreams are best identified by blast radius: broad installation footprint, long-lived archival data, domain-standard file conventions, or use as an intermediate layer for other tools. The following categories of downstreams are especially relevant.

Scientific data models and exchange formats.
The highest-impact downstream is netCDF-4, which uses HDF5 as the storage layer for the enhanced netCDF data model and is central to climate, weather, ocean, atmospheric, and other Earth-system data workflows [32]. NASA HDF-EOS5, NeXus, and CGNS similarly define domain-specific conventions over HDF5 for Earth observation, neutron/x-ray/muon facilities, and CFD or aerospace simulation data [17, 26, 7]. For these formats, HDF5 compatibility is not just a library question: it affects the readability of long-lived archives and the ability of independent tools to interpret a shared schema.

Language bindings and analysis libraries.
h5py, PyTables, pandas HDFStore, Bioconductor's rhdf5, HDF5.jl, and MATLAB's HDF5 interfaces make HDF5 a routine dependency for Python, R, Julia, and MATLAB users [16, 29, 28, 6, 20, 25]. These packages are important because they often sit at the boundary between upstream HDF5 builds and end-user scripts. Their packaging choices determine whether users load a bundled HDF5, a system HDF5, or an environment-provided HDF5, and therefore whether ABI, plugin-path, thread-safety, MPI, and filter support match the files being opened.

Geospatial, visualization, and inspection tools.
GDAL's HDF5 driver, ParaView/VTK-family builds, ITK's HDF5 module, HDFView, and similar readers expose HDF5 files to interactive and automated analysis workflows [27, 21, 19]. This is an important security boundary: these tools are commonly used to inspect third-party data files, so robustness against malformed HDF5 objects, unusual filters, and unexpected VFD/plugin configuration has direct operational value.

HPC and parallel-I/O frameworks.
ADIOS2 includes an HDF5 engine, while netCDF-4, CGNS, h5py, and many simulation applications can be built against serial or parallel HDF5 variants [1]. These downstreams amplify configuration risk: MPI ABI compatibility, collective I/O semantics, file locking behavior, subfiling support, and optional VFDs must line up across the application, HDF5, MPI, and filesystem stack.

Machine-learning and model-artifact workflows.
TensorFlow/Keras still supports older HDF5 model and weight files even as newer native formats are preferred [30]. This keeps HDF5 in the model-exchange and training-checkpoint ecosystem, usually through h5py, and makes HDF5 parser behavior relevant for model files obtained from external sources.

Table 14 – Representative downstream projects that depend on HDF5.

Downstream	Role	Supply-chain relevance

netCDF-4	Common scientific data model and format layered on HDF5.	Large transitive footprint through climate, weather, ocean, remote-sensing, and HPC data stacks. Uses a constrained HDF5 subset, so compatibility depends on both HDF5 behavior and netCDF conventions.
HDF-EOS5	NASA Earth-observation format built on HDF5 concepts and libraries.	Long-lived public archives and many third-party readers make backward compatibility and robust metadata handling high-value.
NeXus	Facility data convention for neutron, x-ray, and muon science.	Uses HDF5 groups, datasets, and attributes as a standardized experimental-data container, so HDF5 schema interpretation affects reproducibility across facilities.
CGNS	CFD and aerospace data standard with an HDF5 mapping.	Important for mesh and simulation-result exchange; parallel-HDF5 behavior and large-file support affect production simulation workflows.
h5py, PyTables, and pandas HDFStore	Python interfaces and tabular/array storage layers.	Common bridge between binary HDF5 files and Python analysis. Wheel and conda packaging can hide which HDF5 build is actually loaded.
rhdf5, HDF5.jl, MATLAB HDF5 APIs	R, Julia, and MATLAB access layers.	Expose HDF5 to laboratory and scientific-computing workflows where files are often exchanged outside controlled build environments.

GDAL and geospatial readers	Raster and subdataset access for HDF5-based products.	Converts HDF5 issues into geospatial ingestion issues, especially for remote-sensing products with complex metadata and nested subdatasets.
ParaView/VTK, ITK, HDFView, and viewers	Interactive inspection, visualization, and conversion.	Frequently open untrusted or externally generated files; defensive parsing, filter availability, and plugin search paths matter.
ADIOS2 and simulation I/O layers	Alternative I/O stacks with HDF5 read/write engines or compatibility paths.	Parallel and high-throughput workflows depend on matching MPI, filesystem, and HDF5 feature assumptions across the whole stack.
TensorFlow/Keras HDF5 artifacts	Legacy model and weight serialization.	Keeps HDF5 in ML artifact exchange; malformed or incompatible model files surface through h5py and HDF5 library behavior.

6.3. Audit Findings and Mitigation Priorities

- HDF5 changes have a large transitive blast radius because many users reach HDF5 through netCDF, GDAL, h5py, MATLAB, or domain formats rather than through the HDF5 C API directly.
- ABI and packaging mismatches are a recurring downstream risk. Python, R, Julia, MATLAB, MPI, and system-package environments may all load different HDF5 builds in the same organization.
- File parsing is a practical security boundary. Viewers, converters, notebooks, and data-ingestion jobs often process files received from outside the organization that built the HDF5 library.

- Optional features can become hidden interoperability requirements: filter plugins, SZIP/zlib variants, ROS3/HDFS/VFD support, parallel HDF5, and file-locking behavior may be assumed by downstream data even when they are not present in a default HDF5 build.
- Long-lived scientific archives make deprecation slower than ordinary application software. Even when a downstream project prefers a newer format, historic HDF5 files remain operationally relevant.

7. Independent HDF5 Implementations

7.1. Overview

- HDF5 library playing footsie with the file format spec: GitHub issue 6409 [2]

PureHDF Managed .NET read/write without native binaries

jHDF JVM HDF5 reading

netCDF-Java JVM netCDF/HDF-EOS/HDF5 scientific-data reading

pyfive Pure Python read-only inspection

jsfive Browser-side direct HDF5 read

scigolib/hdf5 Pure Go read/write

hdf5-pure and rustyhdf5-format/rustyhdf5 Pure Rust read/write

async-hdf5 or h5coro Cloud object-store chunk mapping

JLD2.jl Julia-native persistence in HDF5-compatible files

7.2. Audit Findings and Mitigation Priorities

The main risk across all direct-format implementations is not opening simple files; it is coverage of the long tail: dense groups, fractal heaps, shared object-header messages, all chunk-index variants, references, VLEN, compound/nested datatypes, virtual datasets, external storage, SWMR flags, user blocks, non-core filters, unusual libver bounds, and files produced by different HDF5 library releases. For any production use, build a compatibility corpus and round-trip the generated files using `h5dump`, `h5debug`, `h5py/libhdf5`, and the target-independent implementation.

Machine-readable specification: `hdf5-pickles`

8. Long-term Data Accessibility and Interpretability

Long term data accessibility and interpretability [4]

8.1. Audit Findings and Mitigation Priorities

9. Resources

- HDF SSP SIG [15]
- Tools
  - HDF5 CVE test suite [12]
  - A machine-readable specification of the HDF5 file format. [8]
  - HDF5 extension templates [13]
  - HDF5 plugins [14]
- S2D2

References

[1] ADIOS2 Developers. *ADIOS2 Supported Engines: HDF5*. 2026. URL: https://adios2.readthedocs.io/en/latest/engines/engines.html#hdf5. (accessed: 29.05.2026).

[2] Apollo3zehn. *HDF5 lib is more restrictive than file format spec*. 2026. URL: https://github.com/HDFGroup/hdf5/issues/6409. (accessed: 21.05.2026).

[3] Multiple authors. *179 results for 'file corruption'*. 2007-2026. URL: https://forum.hdfgroup.org/search?q=file%20corruption. (accessed: 21.05.2026).

[4] Multiple authors. *223 results for 'plugin'*. 2007-2026. URL: https://forum.hdfgroup.org/search?q=plugin. (accessed: 21.05.2026).

[5] Andreas Beckmann. *How to properly close HDF files when disk is full?* 2023. URL: https://forum.hdfgroup.org/t/how-to-properly-close-hdf-files-when-disk-is-full/11714. (accessed: 21.05.2026).

[6] Bioconductor Project. *rhdf5*. 2026. URL: https://bioconductor.org/packages/rhdf5. (accessed: 29.05.2026).

[7] CGNS Steering Committee. *CGNS/HDF5 – An HPC implementation*. 2026. URL: https://cgns.org/standard/hdf5.html. (accessed: 29.05.2026).

[8] The HDF Group. *A machine-readable specification of the HDF5 file format*. 2026. URL: https://github.com/HDFGroup/hdf5-pickles. (accessed: 21.05.2026).

[9] The HDF Group. *Commit 3914bb7*. Sep 25, 2025. URL: https://github.com/HDFGroup/hdf5/commit/3914bb7f7ec7105d8bfbeb3aebd92e867cff5 (accessed: 21.05.2026).

[10] The HDF Group. *Commit 804f3bae*. Aug 13, 2025. URL: https://github.com/HDFGroup/hdf5/commit/804f3bace997e416917b235dbd3beac3652a8 (accessed: 21.05.2026).

[11] The HDF Group. *Commit ce53bc0*. Mar 31, 2024. URL: https://github.com/HDFGroup/hdf5/commit/ce53bc020d37e10a06a59501c45f1ab82539f (accessed: 21.05.2026).

[12] The HDF Group. *HDF5 CVE Test Suite*. 2026. URL: https://github.com/HDFGroup/cve_hdf5. (accessed: 21.05.2026).

[13] The HDF Group. *HDF5 extension skeletons, literally*. 2026. URL: https://github.com/HDFGroup/hdf5-boneyard. (accessed: 21.05.2026).

[14] The HDF Group. *HDF5 Plugins*. 2026. URL: https://github.com/HDFGroup/hdf5_plugins. (accessed: 21.05.2026).

[15] The HDF Group. *HDF5 Safety, Security & Privacy (SSP) Special Interest Group's governance, processes, and artifacts*. 2026. URL: https://github.com/HDFGroup/hdf5-ssp-sig. (accessed: 21.05.2026).

[16] h5py Developers. *HDF5 for Python*. 2026. URL: https://docs.h5py.org/. (accessed: 29.05.2026).

[17] HDF-EOS Tools and Information Center. *HDF-EOS Libraries*. 2026. URL: https://hdfeos.org/software/library.php. (accessed: 29.05.2026).

[18] Zero Day Initiative. *CVE-2026-2492*. 2026. URL: https://www.cve.org/CVERecord?id=CVE-2026-2492. (accessed: 21.05.2026).

[19] Insight Software Consortium. *ITK Module: ITKHDF5*. 2026. URL: https://docs.itk.org/projects/doxygen/en/stable/group__ITKHDF5.html. (accessed: 29.05.2026).

[20] JuliaIO. *HDF5.jl*. 2026. URL: https://juliaio.github.io/HDF5.jl/dev/. (accessed: 29.05.2026).

[21] Kitware, Inc. *Building ParaView*. 2026. URL: https://www.paraview.org/paraview-docs/latest/cxx/md__builds_gitlab-kitware-sciviz-ci_Documentation_dev_build.html. (accessed: 29.05.2026).

[22] Google LLC. *CVE-2025-9905*. 2025. URL: https://www.cve.org/CVERecord?id=CVE-2025-99057. (accessed: 21.05.2026).

[23] Google LLC. *CVE-2026-0897*. 2026. URL: https://www.cve.org/CVERecord?id=CVE-2026-0897. (accessed: 21.05.2026).

[24] Google LLC. *CVE-2026-1669*. 2026. URL: https://www.cve.org/CVERecord?id=CVE-2026-1669. (accessed: 21.05.2026).

[25] MathWorks. *HDF5 Files*. 2026. URL: https://www.mathworks.com/help/matlab/hdf5-files.html. (accessed: 29.05.2026).

[26] NeXus International Advisory Committee. *NeXus Data Format Manual*. 2026. URL: https://manual.nexusformat.org/. (accessed: 29.05.2026).

[27] Open Source Geospatial Foundation. *HDF5 – Hierarchical Data Format Release 5 (HDF5)*. 2026. URL: https://gdal.org/en/stable/drivers/raster/hdf5.html. (accessed: 29.05.2026).

[28] pandas Developers. *IO tools: HDF5 / PyTables*. 2026. URL: https://pandas.pydata.org/docs/dev/user_guide/io.html#hdf5-pytables. (accessed: 29.05.2026).

[29] PyTables Developers. *PyTables Documentation*. 2026. URL: https://www.pytables.org/. (accessed: 29.05.2026).

[30] TensorFlow Authors. *Save, serialize, and export models*. 2026. URL: https://www.tensorflow.org/guide/keras/save_and_serialize. (accessed: 29.05.2026).

[31] The HDF Group. *HDF5 2.0.0 source-artifact integrity check*. URL: https://github.com/HDFGroup/hdf5-ssp-sig/tree/main/audit/proofs/hdf5-core/2.0.0/psirt/release-integrity/PRISUPP-2204. (accessed: 03.05.2026).

[32] Unidata Program Center. *NetCDF-4 File Format*. 2026. URL: https://docs.unidata.ucar.edu/nug/2.0-draft/nc4_file_format.html. (accessed: 29.05.2026).

A. CVE History Summary

Table 15 – HDF5 CVE history summary

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2025-7069, CVE-2025-6270, CVE-2025-2914	file-space info / free-space manager	heap buffer overflow	file-derived file-space metadata, especially page size / section data, was accepted without sufficient sanity checks. Fix rejects invalid page size and corrupted free-space info early.
CVE-2025-6858, CVE-2025-6817, CVE-2025-2913, CVE-2025-2912	object-header / metadata-cache cascade	null dereference, resource consumption, use-after-free, heap overflow	corrupted object-header continuation metadata caused later cache/free-list/object-message failures. Root cause is bad continuation-message size/state not rejected at decode time.
CVE-2025-6856, CVE-2025-6816, CVE-2025-2923	object-header continuation messages	heap buffer overflow / use-after-free	crafted object header with continuation message pointing back to itself led to under-sized internal buffer allocation. Fix compares expected and actual object-header chunk counts during deserialization.
CVE-2025-6750, CVE-2024-33874	mtime object-header messages	heap buffer overflow	old/new modification-time messages were not properly decoded; an invalid message size could produce a zero-size buffer passed to an encoder. Fix decodes both mtime formats and rejects invalid size.
CVE-2025-6269, CVE-2025-6516	metadata cache image / address decode	heap buffer overflow	cache-image reconstruction used insufficient bounds and input validation. Fix adds buffer-size tracking, overflow checks, input validation, and cleanup on failure.
CVE-2025-44905, CVE-2025-2308, CVE-2024-29159, CVE-2024-32615, CVE-2016-4331	scale-offset / n-bit filters	heap overflow / buffer overrun / arbitrary code execution	filter decoders trusted encoded precision, byte counts, or scale-offset parameters from the file. Root cause is insufficient bounds checking in decompression/filter parameter handling.
CVE-2025-44904, CVE-2024-32614, CVE-2024-32605, CVE-2024-29165, CVE-2019-9151, CVE-2018-14035, CVE-2018-13875, CVE-2018-11205	vectorized memory copy / scatter-fill paths	heap/stack overflow or overread	memory-vector copy routines copied between mismatched source/destination selections or trusted element counts too far. Root cause is insufficient validation of vector extents and copy sizes before `memcpy`-style operations.
CVE-2025-2926	object-header cache checksum serialization	null pointer dereference	object-header checksum/serialization path could operate on incomplete or invalid object-header state. Root cause is missing null/state validation before serialization.
CVE-2025-2925	memory management wrapper	double free	reallocation/error path mishandled ownership of `mem`. Root cause is ambiguous ownership/cleanup semantics around failed or edge-case realloc.
CVE-2025-2924, CVE-2024-32613, CVE-2024-32612	local heap free-list deserialization	heap buffer overflow	local-heap free-block metadata from the file was trusted. Root cause is insufficient validation of free-block size/offset before rebuilding heap structures.

Table 15 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2025-2915, CVE-2018-13867	file accumulator	heap overflow / out-of-bounds read	accumulator overlap/offset math accepted invalid ranges. Root cause is weak range validation for file accumulator reads/frees.
CVE-2025-2310, CVE-2019-9152	datatype / attribute string decoding	heap buffer overflow / out-of-bounds read	string duplication from metadata used file-derived length or unterminated data. Root cause is unsafe string length handling during datatype/attribute decode.
CVE-2025-2309, CVE-2024-29163	datatype bit operations	heap buffer overflow	bit-copy / bit-find logic trusted bit offsets, lengths, or precision metadata. Root cause is insufficient bounds checking in datatype bitfield operations.
CVE-2025-2153	shared-message deletion	heap buffer overflow	shared-message deletion path could process corrupted shared-message/object-header state. Root cause appears to be insufficient validation of shared-message metadata and cleanup state before deletion.
CVE-2025-26200, CVE-2024-33877, CVE-2017-17507	compound datatype conversion	heap overflow / out-of-bounds read/write	compound datatype conversion trusted complex datatype layout details. Root cause is unsafe datatype conversion when member layout, unused bits, or conversion metadata are malformed.
CVE-2024-33876	point selection deserialization	heap overflow	serialized point-selection metadata was trusted. Root cause is missing bounds/count validation when rebuilding point selections.
CVE-2024-33875, CVE-2019-8396	dataset layout encode	heap/buffer overflow	dataset layout metadata could encode inconsistent dimensions/storage sizes. Fix family adds dataset layout size checks, multiplication overflow checks, and file-bound checks.
CVE-2024-33873	dataset scatter memory	heap overflow	scatter operation accepted inconsistent memory/file selection sizes. Root cause is insufficient validation of selected element counts and destination capacity.
CVE-2024-32624, CVE-2024-32610	reference datatype memory nulling	heap overflow	reference-memory initialization/nulling used malformed datatype/reference metadata. Root cause is missing size/type validation before clearing reference memory.
CVE-2024-32623	generic array fill	heap overflow	array fill operation trusted element count/size metadata. Root cause is missing multiplication/range validation before filling memory.
CVE-2024-32622, CVE-2024-29158, CVE-2024-29161 note	free-list array allocation	out-of-bounds read / stack overflow	array allocation/free-list path accepted invalid counts or sizes. Root cause is weak validation of allocation quantity derived from file metadata.
CVE-2024-32621, CVE-2024-29162, CVE-2024-29157	global heap read	heap/stack overflow	global heap object size/offset metadata could be inconsistent with actual buffer. Root cause is insufficient validation of heap object boundaries before read.
CVE-2024-32620, CVE-2025-6516, CVE-2021-45830, CVE-2018-13866	address-length decode	heap/stack overflow / over-read	encoded file addresses/lengths could exceed expected bounds. Root cause is insufficient validation of decoded address/length byte counts and destination capacity.

Table 15 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2024-32619, CVE-2018-14031	datatype copy/reopen	heap overflow / over-read	datatype copy path accepted malformed nested datatype metadata. Root cause is missing validation while copying or reopening datatype structures.
CVE-2024-32618	native datatype conversion	heap overflow	native-type construction trusted malformed datatype metadata. Root cause is inadequate validation during recursive datatype normalization.
CVE-2024-32617	`H5MM_xstrdup`	heap overflow	unsafe string duplication from file-derived metadata. Root cause is missing bounded string handling.
CVE-2024-32616	datatype object-header encode helper	heap overflow	datatype message encode helper accepted inconsistent datatype-message size/state. Root cause is missing encode-time size validation.
CVE-2024-32611	attribute release table	uninitialized value / DoS	attribute-table bookkeeping conflated allocated capacity and actual attribute count. Fix separates current count and maximum capacity and cleans partial tables on error.
CVE-2024-32608, CVE-2024-32607	attribute close	memory corruption / segfault	attribute close/release path could operate on partially initialized or corrupted attribute state. Root cause is unsafe cleanup after failed attribute-table construction or decode.
CVE-2024-32606	h5tools string printing	uninitialized dereference / DoS	tool-side string formatting could dereference uninitialized values. No commit URL listed, so fix review was not possible.
CVE-2024-29166	link-info decode	buffer overrun	object link-info metadata decode trusted malformed encoded size/flags. No commit URL listed.
CVE-2022-26061, CVE-2022-25972, CVE-2022-25942, CVE-2020-10809, CVE-2018-17436, CVE-2018-17433	`gif2h5` tool / GIF parser	heap overflow, out-of-bounds write/read, invalid write	legacy GIF conversion code parsed attacker-controlled GIF/HDF inputs unsafely. Commentary says the tool was removed rather than surgically hardened.
CVE-2021-46244	datatype copy completion	divide by zero / DoS	datatype copy logic failed to guard arithmetic denominators or zero-sized datatype state.
CVE-2021-46243	datatype object-header decode	untrusted pointer dereference / DoS	datatype decode accepted malformed pointer/metadata state. Root cause is missing validation before dereference.
CVE-2021-46242, CVE-2020-10810	metadata cache pin/unpin	use-after-free / null dereference	cache entry lifecycle operations did not robustly handle malformed or unexpected cache state.
CVE-2021-45833	chunk file-map hyperslab	stack overflow / DoS	chunk/hyperslab mapping trusted malformed dataspace/chunk metadata. Root cause is inadequate bounds checking in chunk mapping.
CVE-2021-45832	error stack internals	stack overflow / DoS	listed as cannot reproduce; no fix commit. Treat root cause as unconfirmed.
CVE-2021-45829	unspecified HDF5 path	segfault / DoS	Root cause undetermined.
CVE-2021-37501	`h5dump` string sprint	buffer overflow / DoS	h5dump formatting path wrote beyond buffer while rendering file-derived data.
CVE-2021-36977	mat file I/O library using HDF5	heap overflow	not filed against HDF5. Root cause belongs to downstream integration/context, not the HDF5 fix set.

Table 15 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2021-31009	Apple platform issue	multiple HDF5 issues	not filed against HDF5; fixed by Apple by removing HDF5 from affected components.
CVE-2020-18494, CVE-2020-18232	dataspace close	buffer overflow / possible RCE	malformed file could corrupt dataspace cleanup/close state. Root cause is unsafe lifecycle handling of malformed dataspace metadata.
CVE-2020-10812	file reference count query	null pointer dereference / DoS	file query path failed to validate file/reference state before dereference.
CVE-2020-10811, CVE-2018-14033	layout decode	heap over-read	dataset layout message decoder trusted encoded layout fields. Root cause is missing bounds checks around decoded layout data.
CVE-2019-8398, CVE-2019-8397	datatype size/close	out-of-bounds read	datatype object state could be malformed before size/close operations. No commit URL listed.
CVE-2018-17439	dataspace extent dimensions during HDF to GIF conversion	stack overflow	conversion tool path trusted dataspace dimensions from file.
CVE-2018-17438	selected I/O	divide by zero / DoS	selected-I/O path failed to guard divisor/count metadata from crafted file. Tool-removal commit, so the listed fix is coarse.
CVE-2018-17437	datatype decode helper	memory leak / DoS	error path leaked allocations when datatype metadata was malformed.
CVE-2018-17435	attribute decode	heap over-read	attribute message decoder trusted encoded sizes/offsets.
CVE-2018-17434	h5repack filter application	divide by zero / DoS	tool-side filter setup failed to guard zero-valued filter/chunk parameters.
CVE-2018-17432	simple dataspace encode	null pointer dereference / DoS	dataspace encode path accepted invalid dataspace state before dereference.
CVE-2018-17237, CVE-2018-17233, CVE-2018-11207, CVE-2018-11203, CVE-2017-17508	chunking / b-tree / datatype arithmetic	divide by zero / SIGFPE / DoS	crafted file could set chunk, b-tree, datatype, or layout parameters to zero. Root cause is missing arithmetic precondition checks.
CVE-2018-17234, CVE-2018-13873, CVE-2018-11204	object-header chunk deserialize	memory leak / over-read / null dereference	object-header chunk deserialization accepted malformed chunk metadata and mishandled error cleanup.
CVE-2018-16438	external links	out-of-bounds read	external-link query trusted malformed external-link metadata.
CVE-2018-15672, CVE-2018-14032	duplicate/rejected IDs	not applicable	rejected duplicate CVEs; no HDF5 root cause to summarize beyond the referenced canonical CVEs.
CVE-2018-15671	property-list callback lookup	excessive stack consumption / DoS	recursive or malformed property-list callback state allowed stack exhaustion.
CVE-2018-14460	simple dataspace decode	heap over-read	dataspace message decoder trusted encoded extent/rank metadata.
CVE-2018-14034	filter pipeline reset	out-of-bounds read	malformed filter-pipeline metadata caused unsafe reset/read behavior. No commit URL listed.

Table 15 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary
CVE-2018-13876, CVE-2018-13874	sec2 VFD read path	stack overflow	low-level file-driver read/memset path could be driven out of bounds by malformed file metadata. No commit URL listed.
CVE-2018-13872	group entry decode	heap overflow	group-entry decoder trusted malformed symbol-table/group metadata. No commit URL listed.
CVE-2018-13871	free-list block allocation	heap overflow	free-list block allocator accepted invalid allocation size/count.
CVE-2018-13870, CVE-2018-13869	link decode	heap over-read / overlapping memcpy	link-message decoder trusted malformed link metadata, including overlapping source/destination regions.
CVE-2018-13868	old fill-value decode	heap over-read	old fill-value decoder trusted encoded fill-value size. No commit URL listed.
CVE-2018-11206	fill-value decode	out-of-bounds read / possible disclosure	fill-value message decoders trusted encoded size/type fields.
CVE-2018-11202	hyperslab spans	null pointer dereference / DoS	hyperslab span construction failed to validate malformed selection state.
CVE-2017-17509	group cache entry decode vector	out-of-bounds write	group-entry vector decoder trusted file-derived counts/array bounds. No commit URL listed.
CVE-2017-17506, CVE-2017-17505	filter pipeline decode	out-of-bounds read / null dereference	pipeline decoder failed to validate malformed filter counts or fields. No commit URL listed.
CVE-2016-4333	array allocation / initialization loop	out-of-bounds write	file-controlled value could modify loop termination while initializing an allocated array.
CVE-2016-4332	object-header message flags	heap out-of-bounds write / code execution	library failed to check whether a message type supported a flag before casting to an incompatible structure.
CVE-2016-4330	dataspace array rank/dimensions	heap overflow / code execution	file-supplied array dimensionality was not checked against allocated space.

B. Software Supply Chain Details

The HDF5_ALLOW_EXTERNAL_SUPPORT modes apply to filter-related dependencies. Some test and example configurations have their own source-fetch paths, such as KWSys for the API test driver and h5py for Python examples. MPI, HDFS, ROS3, OpenSSL, Java, documentation, packaging, and developer tools must be provided by the build environment.

Table 16 – Dependency source modes

Mode	Relevant options	Meaning
System packages	HDF5_ALLOW_EXTERNAL_SUPPORT=NO	CMake uses find_package(), find_library(), or find_program() to locate installed dependencies. This is the default.
Git fetch	HDF5_ALLOW_EXTERNAL_SUPPORT=GIT plus a dependency-specific *_USE_EXTERNAL=ON option	CMake downloads source from the configured *_GIT_URL and *_GIT_TAG values and builds it with HDF5.
Tarball fetch	HDF5_ALLOW_EXTERNAL_SUPPORT=TGZ plus a dependency-specific *_USE_EXTERNAL=ON option	CMake downloads from *_TGZ_ORIGPATH and *_TGZ_NAME, or uses TGZPATH when the dependency-specific *_USE_LOCALCONTENT=ON option is set.

Table 17 – Baseline build

Dependency	Trigger	Notes
CMake 3.26 or newer	Always	Required by the top-level cmake_minimum_required().
C compiler and native build tool	Always	The top-level project is a C project.
Platform C library, headers, and system calls	Always	CMake probes standard headers, types, functions, byte order, large-file support, file locking, pread/pwrite, and related platform features.
Platform link libraries	Platform-dependent	CMake adds libraries only when probes show they are needed: m, dl, rt, posix4, ucb, ws2_32, wsock32, and shlwapi.

PkgConfig	Optional probe	Used only when available. It can help some find modules, such as zlib-ng and libaec, but it is not a hard requirement for the default build.
Threads	Found in every configure; required by thread features	CMake searches for C11 threads, Win32 threads, or pthreads. When found, HDF5 records thread support and links `Threads::Threads`; `HDF5_ENABLE_THREADSAFE`, `HDF5_ENABLE_CONCURRENCY`, and `HDF5_ENABLE_SUBFILING_VFD` require it.

Table 18 – HDF5 feature triggers

Feature or component	Build options	Dependency	Notes
Parallel HDF5	`HDF5_ENABLE_PARALLEL=ON`	MPI C with MPI-IO, MPI standard 3.0 or newer	Adds `MPI::MPI_C` to public link libraries. Parallel filtered writes and large parallel I/O are enabled only when the required MPI symbols are present. Parallel tests also need an MPI launcher such as `mpiexec`.
Parallel Fortran	`HDF5_ENABLE_PARALLEL=ON`, `HDF5_BUILD_FORTRAN=ON`	MPI Fortran	The Fortran build adds MPI Fortran libraries. The `mpi_f08` module is used when available.
Thread-safe library	`HDF5_ENABLE_THREADSAFE=ON`	Threads	Unsupported with parallel, Fortran, C++, or high-level library unless `HDF5_ALLOW_UNSUPPORTED=ON`. On Windows, static libraries are rejected.

Multi-threaded concurrency	HDF5_ENABLE_CONCURRENCY=ON	Threads	Experimental. Mutually exclusive with HDF5_ENABLE_THREADSAFE; has the same unsupported combinations as thread-safety unless overridden with HDF5_ALLOW_UNSUPPORTED=ON.
C++ wrappers	HDF5_BUILD_CPP_LIB=ON	C++ compiler	Unsupported with parallel unless HDF5_ALLOW_UNSUPPORTED=ON. High-level C++ wrappers also require HDF5_BUILD_HL_LIB=ON.
Fortran wrappers	HDF5_BUILD_FORTRAN=ON	Fortran compiler with Fortran 2003 ISO_C_BINDING support	Unsupported with thread-safety or concurrency unless HDF5_ALLOW_UNSUPPORTED=ON. High-level Fortran wrappers also require HDF5_BUILD_HL_LIB=ON.
Java wrappers	HDF5_BUILD_JAVA=ON	Java/JDK, shared HDF5 libraries, Java dependency JARs	Java requires shared HDF5 libraries. Java 11 or newer is required for the JNI implementation. Java tests additionally require the JUnit and Hamcrest JARs.
Java JNI implementation	HDF5_BUILD_JAVA=ON, HDF5_ENABLE_JNI=ON, or any Java version earlier than 25	JNI headers and libraries	JNI is the default Java implementation. HDFS also requires JNI.
Java FFM implementation	HDF5_BUILD_JAVA=ON, Java 25 or newer, HDF5_ENABLE_JNI=OFF	jextract from JEXTRACT_HOME/bin or JAVA_HOME/bin	Generates FFM bindings at configure time.

Java logging/test JARs	`HDF5_BUILD_JAVA=ON`	`slf4j-api`, `slf4j-nop`, `slf4j-simple`; plus JUnit 4 and Hamcrest for tests	The default paths point to bundled JARs under `java/lib`. They can be overridden with `HDF5_JAVA_*_JAR` cache variables. `HDF5_ENABLE_MAVEN_DEPLOY=ON` generates Maven-style artifacts and POM files; the CMake build does not locate or run `mvn`.
Documentation	`HDF5_BUILD_DOC=ON`	Doxygen	`HDF5_ENABLE_DOXY_WARNINGS=ON` makes Doxygen warnings fail the build.
Header regeneration	`HDF5_GENERATE_HEADERS=ON`	Perl	Perl is optional otherwise. If it is missing, generated headers are not regenerated.
Python examples	`H5EXAMPLE_BUILD_PYTHON=ON`	Python3 interpreter, Python development files, NumPy, h5py source, pip	The examples CMake project fetches `h5py` from Git and installs it with `python3 -m pip`.

Table 19 – HDF5 filter dependency choices

Feature	Build options	Dependency	Source choices
DEFLATE filter	`HDF5_ENABLE_ZLIB_SUPPORT=ON`, `HDF5_USE_ZLIB_NG=OFF`	zlib	System package by default; `ZLIB_USE_EXTERNAL=ON` can build zlib from Git or TGZ. `HDF5_USE_ZLIB_STATIC=ON` prefers a static library.

DEFLATE filter through zlib-ng	`HDF5_ENABLE_ZLIB_SUPPORT=ON`, `HDF5_USE_ZLIB_NG=ON`	zlib-ng	System package by default; `ZLIB_USE_EXTERNAL=ON` can build zlib-ng from Git or TGZ. zlib compatibility mode is detected.
SZIP filter	`HDF5_ENABLE_SZIP_SUPPORT=ON`	libaec with libsz compatibility, or another package selected by `LIBAEC_PACKAGE_NAME`	System package by default; `SZIP_USE_EXTERNAL=ON` can build libaec from Git or TGZ. `HDF5_USE_LIBAEC_STATIC=ON` prefers static libraries.
HDF5 filter plugins project	`HDF5_ENABLE_PLUGIN_SUPPORT=ON`	Installed `h5pl`/HDF5 filter plugins package, or the `hdf5_plugins` project	`PLUGIN_USE_EXTERNAL=ON` can build the plugin project from Git or TGZ. Plugin support is disabled for the 1.6 API.

Table 20 – HDF5 VFD and VOL connector dependency choices

Feature	Build options	Dependency	Notes
Direct VFD	`HDF5_ENABLE_DIRECT_VFD=ON`	POSIX `O_DIRECT` and `posix_memalign()` support	No third-party library. Configuration fails if the platform support is missing.
Mirror VFD	`HDF5_ENABLE_MIRROR_VFD=ON`	POSIX socket/fork headers and functions	No third-party library. Requires `netinet/in.h`, `netdb.h`, `arpa/inet.h`, `sys/socket.h`, and `fork()`.

ROS3 VFD	`HDF5_ENABLE_ROS3_VFD=ON`	`aws-c-s3` package	CMake	HDF5 does not fetch this library. Building `aws-c-s3` from source can also require AWS C libraries such as `aws-c-common`, `aws-checksums`, `aws-c-cal`, `aws-c-io`, `aws-c-compression`, `aws-c-http`, `aws-c-sdkutils`, `aws-c-auth`, and, on Linux, `aws-lc` and `s2n-tls`. ROS3 docker-proxy tests additionally require `docker` and the AWS CLI.
HDFS VFD	`HDF5_ENABLE_HDFS=ON`	JNI/JVM and Hadoop `libhdfs`		`HADOOP_HOME` is used to locate `hdfs.h`, `libhdfs`, and the Hadoop version command. Non-MSVC builds also add `-pthread`.
Subfiling VFD and IOC VFD	`HDF5_ENABLE_PARALLEL=ON`, `HDF5_ENABLE_SUBFILING_VFD=ON`	MPI C with `MPI_Comm_split_type`, plus Threads		Not available on Windows. IOC VFD is built when subfiling is enabled.
External VOL connectors	`HDF5_VOL_ALLOW_EXTERNAL=GIT` or `LOCAL_DIR` with `HDF5_VOL_URLnn` or `HDF5_VOL_PATHnn`	Connector-defined		Up to ten CMake-based VOL connectors can be added. HDF5 patches connector CMake code to use the in-tree HDF5 build, but each connector may bring its own dependencies.

Table 21 – Tools, Tests, and Security Features

Feature	Build options	Dependency	Notes
Signed plugin verification	`HDF5_REQUIRE_SIGNED_PLUGINS=ON`	OpenSSL libcrypto 1.1.0 or newer	Adds `OpenSSL::Crypto` to HDF5. `HDF5_LOCK_PLUGIN_KEYSTORE=ON` also requires `HDF5_PLUGIN_KEYSTORE_DIR`.
`h5sign` tool	`HDF5_REQUIRE_SIGNED_PLUGINS=ON`, `HDF5_BUILD_TOOLS=ON`	OpenSSL libcrypto	Used to sign HDF5 plugins.
Signed-plugin tests	`HDF5_REQUIRE_SIGNED_PLUGINS=ON`, `BUILD_TESTING=ON`	`openssl` executable	Tests generate RSA keys with the OpenSSL command-line tool.
Parallel tools utilities	`HDF5_ENABLE_PARALLEL=ON`, `HDF5_BUILD_PARALLEL_TOOLS=ON`	MFU, CIRCLE, DTCMP	CMake looks for headers and libraries, using `MFU_ROOT` as a hint.
API test driver	`BUILD_TESTING=ON`, `HDF5_TEST_API_ENABLE_DRIVER=ON`	KWSys	The test driver fetches KWSys from a tarball or from `TGZPATH` when `KWSYS_USE_LOCALCONTENT=ON`.
Shell-script tests	`BUILD_TESTING=ON`	PowerShell or Bash	CMake uses `pwsh`/`powershell` when available; otherwise Unix builds look for `bash`.
ROS3 VFD tests	`HDF5_ENABLE_ROS3_VFD=ON`, `HDF5_ENABLE_ROS3_VFD_DOCKER_PROXY=ON`	Docker daemon and AWS CLI	Used to run S3 proxy tests.

Table 22 – Developer and Packaging Dependencies

Configuration	Build options	Dependency
Analyzer tooling	`HDF5_ENABLE_ANALYZER_TOOLS=ON`	`clang-tidy`, `include-what-you-use`, and `cppcheck` are probed and used by the analyzer macros.
Source formatting targets	`HDF5_ENABLE_FORMATTERS=ON`	`clang-format` and `cmake-format`.
Code coverage with Clang/IntelLLVM	`HDF5_ENABLE_COVERAGE=ON`, `CODE_COVERAGE=ON`	`llvm-cov` and `llvm-profdata`; AppleClang uses `xcrun llvm-cov` and `xcrun llvm-profdata`.
Code coverage with GCC	`HDF5_ENABLE_COVERAGE=ON`, `CODE_COVERAGE=ON`	`lcov` and `genhtml`; fallback instrumentation can also link `gcov`.
Sanitizers	`HDF5_ENABLE_SANITIZERS=ON`, `HDF5_USE_SANITIZER=<kind>`	Compiler sanitizer runtime support. Supported values depend on compiler and platform.
AFL support module	Explicit inclusion of `config/sanitizer/afl-fuzzing.cmake`	`afl-cc` and `afl-c++`. This module is available but is not included by the top-level HDF5 build by default.
Dependency graph support module	Explicit inclusion of `config/sanitizer/dependency-graph.cmake`	Graphviz `dot`. This module is available but is not included by the top-level HDF5 build by default.
Windows installers	CPack on Windows	NSIS and WiX are probed when available.
Linux packages	CPack on Unix-like systems	`dpkg-shlibdeps` enables DEB generation; `rpmbuild` enables RPM generation for non-parallel builds.

Table 23 – External Source Defaults

Dependency	Default version/tag	Default source

zlib	1.3.2 / v1.3.2	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.2/zlib-1.3.2.tar.gz
zlib-ng	2.3.3	https://github.com/zlib-ng/zlib-ng.git or https://github.com/zlib-ng/zlib-ng/archive/refs/tags/2.3.3.tar.gz
libaec	1.1.6 / v1.1.6	https://github.com/MathisRosenhauer/libaec.git or https://github.com/MathisRosenhauer/libaec/releases/download/v1.1.6/libaec-1.1.6.tar.gz
HDF5 filter plugins	master	https://github.com/HDFGroup/hdf5_plugins.git or https://github.com/HDFGroup/hdf5_plugins/releases/download/snapshot/hdf5_plugins-master.tar.gz
KWSys API-test helper	master tarball	https://gitlab.kitware.com/utils/kwsys/-/archive/master/kwsys-master.tar.gz
h5py Python examples	master	https://github.com/h5py/h5py.git
bitshuffle	master or 0.5.2 tarball	https://github.com/kiyo-masui/bitshuffle.git or https://github.com/kiyo-masui/bitshuffle/archive/refs/tags/bitshuffle-0.5.2.tar.gz
Blosc	1.21.6	https://github.com/Blosc/c-blosc.git or https://github.com/Blosc/c-blosc/archive/refs/tags/c-blosc-1.21.6.tar.gz
Blosc zlib helper	develop or 1.3.1 tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
Blosc2	2.17.1	https://github.com/Blosc/c-blosc2.git or https://github.com/Blosc/c-blosc2/archive/refs/tags/c-blosc2-2.17.1.tar.gz

Blosc2 zlib helper	`develop` or `1.3.1` tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
bzip2	`bzip2-1.0.8`	https://github.com/libarchive/bzip2.git or https://github.com/libarchive/bzip2/archive/refs/tags/bzip2-bzip2-1.0.8.tar.gz
fpzip	`develop` or `1.3.0` tarball	https://github.com/LLNL/fpzip.git or https://github.com/LLNL/fpzip/releases/download/1.3.0/fpzip-1.3.0.tar.gz
JPEG	`jpeg-9e`	https://github.com/libjpeg-turbo/libjpeg-turbo.git or https://www.ijg.org/files/jpegsrc.v9e.tar.gz
LZ4	`dev` or `1.10.0` tarball	https://github.com/lz4/lz4.git or https://github.com/lz4/lz4/releases/download/v1.10.0/lz4-1.10.0.tar.gz
LZF	`3.6` tarball	http://dist.schmorp.de/liblzf/liblzf-3.6.tar.gz
SZ	`master` or `2.1.12.5` tarball	https://github.com/szcompressor/SZ.git or https://github.com/szcompressor/SZ/releases/download/v2.1.12.5/SZ-2.1.12.5.tar.gz
ZFP	`develop` or `1.0.1` tarball	https://github.com/LLNL/zfp.git or https://github.com/LLNL/zfp/releases/download/1.0.0/zfp-1.0.1.tar.gz
Zstandard	`1.5.7`	https://github.com/facebook/zstd.git or https://github.com/facebook/zstd/releases/download/v1.5.7/zstd-1.5.7.tar.gz